

Sistema de Solución de Problemas de Programación Lineal (Septiembre de 2026)

Juan David Navaro Bermúdez 160005245, Ingeniería de sistemas Sem-5, Universidad de los Llanos, Villavicencio, Colombia, ju.da.navarro@unillanos.edu.co

Resumen - El presente documento describe el diseño e implementación de un software educativo para la resolución de problemas de Programación Lineal (PL) mediante tres métodos fundamentales: el Método Gráfico, el Método Simplex Común (para problemas de maximización) y el Método Simplex Gran M (para problemas de minimización). La aplicación, desarrollada en Python bajo la arquitectura Modelo-Vista-Controlador (MVC), permite modelar problemas con hasta seis variables de decisión y múltiples restricciones, visualizando de forma interactiva y paso a paso el proceso de solución. El informe detalla la arquitectura del sistema, la lógica matemática encapsulada en los solvers y los resultados obtenidos al resolver un caso práctico de prueba.

I. INTRODUCCIÓN

La Programación Lineal es una técnica de optimización ampliamente utilizada en ingeniería, economía y administración para maximizar o minimizar una función lineal sujeta a restricciones lineales. La enseñanza de estos métodos se apoya frecuentemente en herramientas computacionales que permiten visualizar y comprender los algoritmos subyacentes.

Este proyecto implementa un sistema de software que resuelve problemas de PL de forma gráfica (para dos variables) y mediante los métodos Simplex Común y Gran M (para cualquier número de variables). El objetivo es proporcionar una herramienta didáctica basada en la arquitectura MVC que muestre cada iteración del algoritmo, facilitando así el aprendizaje. La interfaz gráfica (desarrollada con Tkinter y Matplotlib) representa la Vista, la cual interactúa con el Controlador para gestionar los datos ingresados y mostrar la evolución de la solución paso a paso producida por el Modelo. El presente informe describe la arquitectura MVC, la estructura de clases y los resultados obtenidos.

II. MARCO TEORICO

2.1 Método Gráfico

El método gráfico es aplicable a problemas con dos variables de decisión. Consiste en representar las restricciones como rectas en el plano cartesiano, identificar la región factible (intersección de todos los semiplanos) y evaluar la función objetivo en los vértices de dicha región. El teorema fundamental de la Programación Lineal establece que, si existe una solución óptima, esta se encuentra en un vértice de la región factible. El método calcula todas las intersecciones de las rectas (incluyendo los ejes de no negatividad) y filtra aquellas que satisfacen todas las restricciones. Finalmente, el vértice que produce el mejor valor de la función objetivo (máximo o mínimo) es la solución óptima.

2.2 Método Simplex Común y Gran M

El método Simplex es un algoritmo algebraico que resuelve problemas de PL con cualquier número de variables. Según las características de las restricciones del problema, el sistema aplica:

Simplex Común: Se utiliza cuando todas las restricciones son de tipo $\leq$ con lados derechos no negativos. Se introducen variables de holgura para formar la base inicial y se itera pivoteando hasta alcanzar la solución óptima.

Simplex Gran M: Se aplica cuando existen restricciones de tipo $\geq$ o de igualdad ($=$). Se introducen variables artificiales penalizadas en la función objetivo con un costo muy grande (M) para forzar su salida de la base durante el proceso de optimización.

Ambos métodos están implementados en las clases GraphicSolver y SimplexModel, respectivamente, y comparten una interfaz común que permite su integración en la aplicación.

III. DESARROLLO DE CONTENIDOS

3.1 Análisis de Requisitos (Historias de Usuario)

Para definir el alcance funcional del sistema desde la perspectiva del usuario final (estudiantes y docentes de

Sistema de Solución de Problemas de Programación Lineal

optimización), se plantearon las siguientes historias de usuario:

ID	Rol	Necesidad / Deseo	Justificación (Propósito)
HU-01	Estudiante	Quiero definir el número de variables de decisión y los coeficientes de la función objetivo.	Para poder modelar correctamente el problema de Programación Lineal que deseo resolver.
HU-02	Estudiante	Quiero agregar, modificar y eliminar restricciones (con operadores $\leq$, $\geq$, $=$) con sus respectivos lados derechos.	Para representar todas las condiciones y limitantes del sistema de inecuaciones del modelo matemático.
HU-03	Estudiante	Quiero seleccionar entre el "Método Gráfico", "Simplex Común" y "Simplex Gran M" para resolver el problema.	Para elegir la técnica adecuada según la cantidad de variables (2 para gráfico, más de 2 para Simplex) y comparar ambos enfoques.
HU-04	Estudiante	Quiero visualizar la solución de forma secuencial, mostrando texto explicativo y gráficos/tablas en cada paso.	Para comprender la lógica interna del algoritmo (cálculo de vértices o pivoteo) y verificar la evolución de la solución.
HU-05	Estudiante	Quiero recibir mensajes de error claros cuando el modelo no tenga región factible o sea no acotado.	Para identificar rápidamente errores en el planteamiento del modelo y entender las limitaciones matemáticas del problema.

3.2 Estructura de Archivos y Organización del Código

Archivo	Responsabilidad	Descripción breve
problem.py	Entidad del problema.	Clase <i>LinearProblem</i> : representa el Modelo en la arquitectura MVC. Almacena coeficientes objetivo, restricciones y tipo de optimización.
graphic_solver.py	Método Gráfico.	Clase <i>GraphicSolver</i> : calcula vértices, evalúa Z y genera figura con región factible y óptimo.
simplex_solver.py	Solver Simplex Común.	Contiene la clase <i>SimplexModel</i> (para el Método Simplex Común), construyendo el tableau y ejecutando su algoritmo.
simple_solver_gran_m.py	Solver Simplex Gran M.	Contiene la clase <i>SimplexModelGranM</i> (para el Método Simplex Gran M), construyendo el tableau y ejecutando su algoritmo con penalizaciones M.
main_view.py	Interfaz gráfica (Vista).	Contiene la <i>class MainView</i> : que define la interfaz gráfica en la arquitectura MVC mediante Tkinter y Matplotlib.
main.py	Controlador principal.	Contiene la <i>class ControladorProgramacionLineal</i> : que actúa como Controlador en MVC, capturando eventos y coordinando la resolución entre la vista y los modelos.

3.3 Diseño del Sistema

El sistema se organiza bajo el patrón arquitectónico Modelo-Vista-Controlador (MVC):

- *LinearProblem*: Clase entidad que almacena los datos del problema: coeficientes de la función objetivo, lista de restricciones (cada una con coeficientes, operador y lado derecho) y tipo de optimización (max/min).
- *GraphicSolver*: Implementa el método gráfico. Recibe una instancia de *LinearProblem* y ofrece métodos para calcular los vértices, evaluar la función objetivo y generar una figura con la región factible.
- *SimplexModel*: Implementa el algoritmo Simplex Común para problemas de maximización con restricciones de tipo $\leq$.
- *SimplexModelGranM*: Implementa el algoritmo del Método de la Gran M para abordar problemas con restricciones de tipo $\geq$ o de igualdad ($=$), manejando penalizaciones M y variables artificiales.

Ambos solvers cumplen con el mismo contrato: el método `resolver()` ejecuta el algoritmo y `obtener_pasos_ui()` devuelve una lista de pasos, cada uno con texto explicativo, una figura (opcional) y un título. Esto permite a la interfaz gráfica mostrar de manera uniforme los resultados, independientemente del método utilizado.

Sistema de Solución de Problemas de Programación Lineal

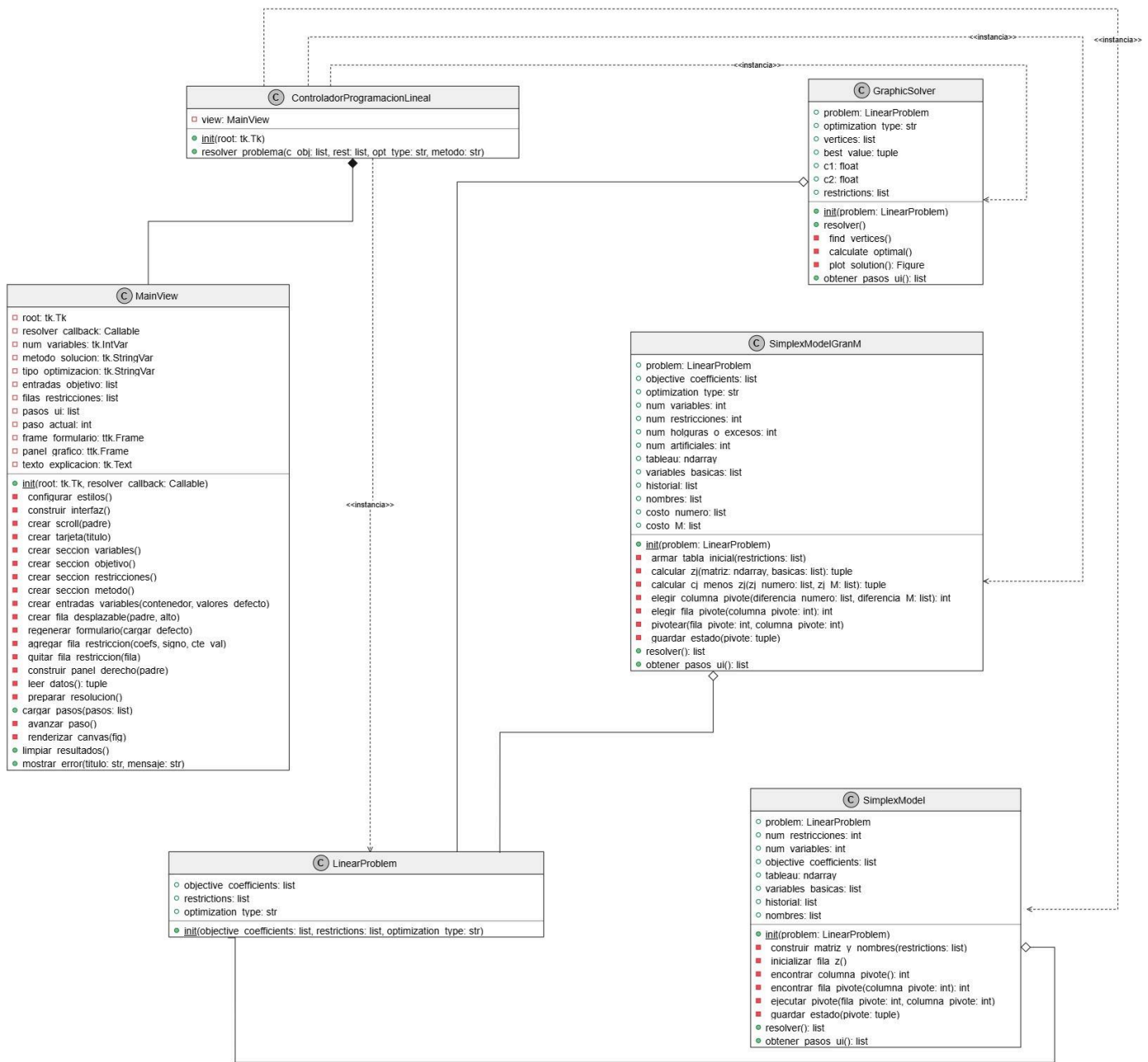


Figura 1: Diagrama de clases relación entre LinearProblem, GraphicSolver y SimplexModel, así como la interfaz

El flujo típico de ejecución es el siguiente:

1. El usuario introduce los datos del modelo a través de la interfaz.
2. Se crea una instancia de **LinearProblem** con los datos ingresados.
3. Según el método seleccionado, se instancia **GraphicSolver** o **SimplexModel**.
4. Se llama a **resolver()**, que realiza los cálculos y almacena internamente los resultados.
5. Se invoca **obtener_pasos_ui()** para recuperar la secuencia de pasos (texto, figuras).
6. La interfaz muestra cada paso de forma secuencial al usuario.

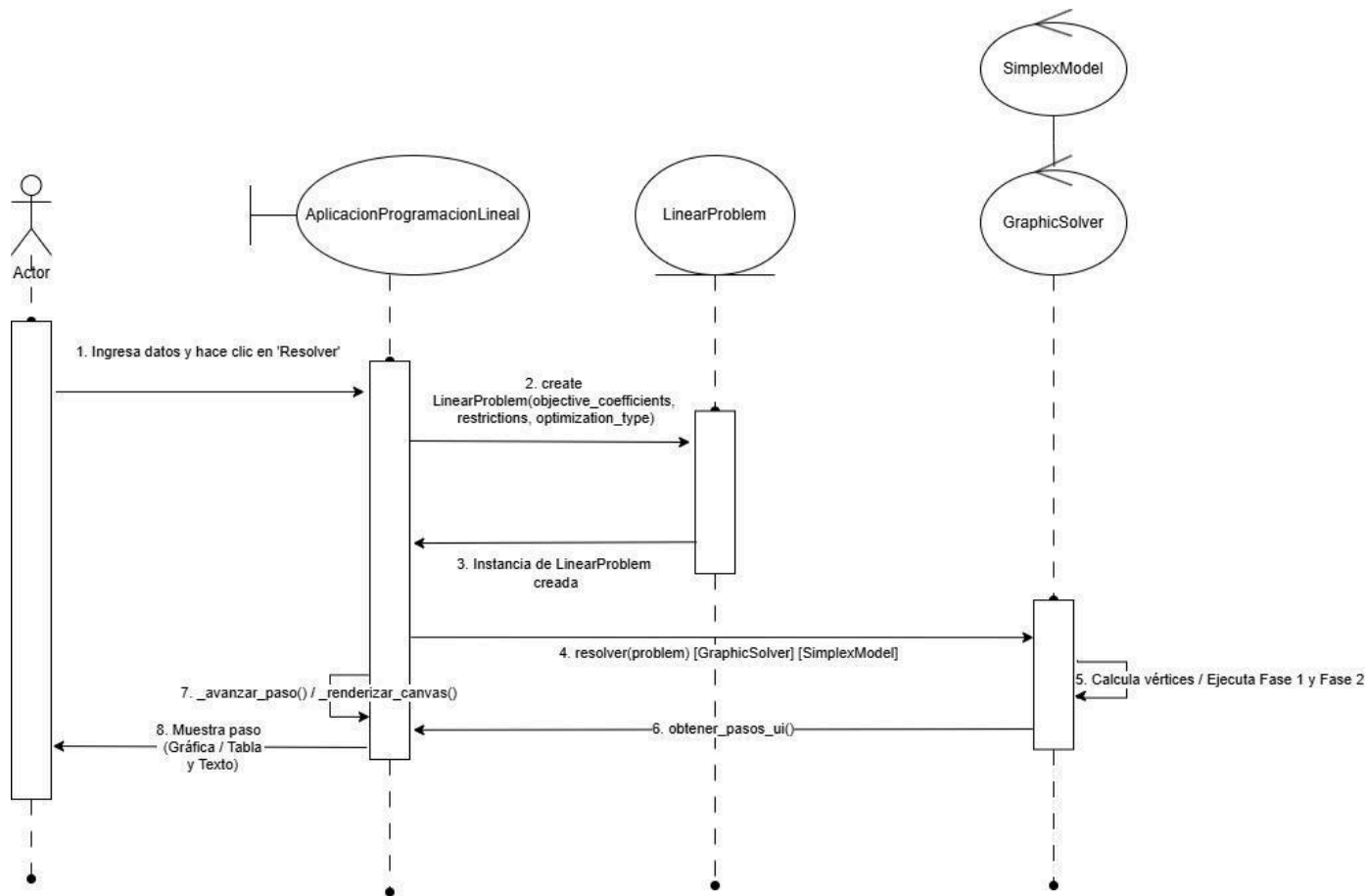


Figura 2: Diagrama de secuencia, proceso de resolución de un problema, desde la entrada de datos hasta la presentación

3.4 Lógica y Funcionamiento del Núcleo (Core)

A continuación, se describen los aspectos más relevantes de la implementación de los dos solvers.

3.4.1 Método Gráfico – GraphicSolver

La clase GraphicSolver se inicializa con un objeto LinearProblem y extrae los coeficientes de la función objetivo (c_1 , c_2) y las restricciones en formato (a , b , operador, rhs). El método principal resolver orquesta dos etapas:

Cálculo de vértices ($find_vertices$):

Se construye una lista de ecuaciones lineales a partir de las restricciones, incluyendo las restricciones de no negatividad $x \geq 0$ e $y \geq 0$ como $(1,0,0)$ y $(0,1,0)$.

Se generan todas las combinaciones de pares de rectas y se resuelve el sistema de dos ecuaciones lineales usando `numpy.linalg.solve`.

Para cada punto solución (x, y) , se verifica que cumpla todas las restricciones originales (incluyendo el operador). Si es factible, se añade al conjunto de vértices.

Los puntos se redondean a 4 decimales para evitar errores de precisión.

```

1  def _find_vertices(self):
2      # Rectas a intersectar: las de las restricciones + los ejes x=0, y=0
3      rectas = [(a, b, c) for a, b, _operador, c in self.restrictions]
4      rectas.append((1, 0, 0)) # eje: x = 0
5      rectas.append((0, 1, 0)) # eje: y = 0
6
7      vertices_validos = set()
8
9      # 2. Intersectar cada par de rectas para obtener posibles vértices.
10     for (a1, b1, c1), (a2, b2, c2) in combinations(rectas, 2):
11
12         # Resolver el sistema 2x2: a1*x + b1*y = c1 ; a2*x + b2*y = c2
13         try:
14             x, y = np.linalg.solve([[a1, b1], [a2, b2]], [c1, c2])
15         except np.linalg.LinAlgError:
16             continue
17
18         x, y = round(x, 4), round(y, 4)
19
20         # Descartar puntos fuera del primer cuadrante.
21         if x < 0 or y < 0:
22             continue
23
24         es_vertice_valido = True
25
26         for a, b, operador, c in self.restrictions:
27             valor_evaluado = round((a * x) + (b * y), 4)
28
29             incumple = (
30                 (operador == "<=" and valor_evaluado > c) or
31                 (operador == ">=" and valor_evaluado < c) or
32                 (operador == "=" and valor_evaluado != c)
33             )
34
35             if incumple:
36                 es_vertice_valido = False
37                 break
38
39         if es_vertice_valido:
40             vertices_validos.add((x, y))
41
42     self.vertices = list(vertices_validos)

```

Figura de código 1: Fragmento de `_find_vertices` mostrando la búsqueda de intersecciones mediante `itertools.combinations` y `np.linalg.solve`.

Evaluación de la función objetivo (`_calculate_optimal`):

Para cada vértice, se calcula $Z = c_1 \cdot x + c_2 \cdot y$.

Según el tipo de optimización (max o min), se selecciona el vértice con el valor extremo. Si no hay vértices, se lanza una excepción indicando región infactible.

La generación de la figura (`_plot_solution`) utiliza matplotlib para dibujar las rectas, sombrear la región factible y marcar los vértices y el óptimo. Esta figura se integra en los pasos de la interfaz.

3.4.2 Método Simplex Común (Maximización)

Sistema de Solución de Problemas de Programación Lineal

La clase SimplexModel ejecuta la resolución mediante el algoritmo Simplex convencional para modelos de maximización cuyas restricciones sean de tipo menor o igual ($\leq$).

Configuración e incorporación de variables (_init_y_construir_matriz_y_nombres):

Confirma la orientación a maximizar y estructura la tabla inicial (tableau), insertando variables de decisión y de holgura (s_i) para convertir desigualdades en ecuaciones.

```

1 class SimplexModel:
2     def __init__(self, problem: LinearProblem):
3         if not isinstance(problem, LinearProblem):
4             raise TypeError("SimplexModel requiere una instancia de LinearProblem")
5
6         if problem.optimization_type != "max":
7             raise ValueError(
8                 "El método Simplex común solo admite maximización. "
9                 "Usa el método de Dos Fases para problemas de Min."
10            )
11
12        self.problem = problem
13        self.restrictions = problem.restrictions
14        self.objective_coefficients = problem.objective_coefficients
15
16        self.num_restricciones = len(restrictions)
17        self.num_variables = len(objective_coefficients)
18        self.objective_coefficients = objective_coefficients
19
20        total_columnas = self.num_variables + self.num_restricciones + 1
21
22        self.tableau = np.zeros((self.num_restricciones + 1, total_columnas))
23        self.variables_basicas = [0] * self.num_restricciones
24        self.historial = []
25
26        self._construir_matriz_y_nombres(restrictions)
27        self._inicializar_fila_z()

```

Verificación de optimalidad (_encontrar_columna_pivote):

Examina el renglón objetivo Z; al ser todos sus valores no negativos, se declara el óptimo. De no ser así, escoge la columna con el término más negativo como columna pivote.

```

1     def _encontrar_columna_pivote(self):
2         fila_z = self.tableau[0, :-1]
3
4         if np.all(fila_z >= 0):
5             return None
6
7         return int(np.argmin(fila_z))

```

Cálculo del cociente mínimo (_encontrar_fila_pivote):

Compara las disponibilidades del lado derecho frente a elementos estrictamente positivos de la columna de entrada, identificando la variable saliente para sostener la factibilidad.

```

1 def _encontrar_fila_pivote(self, columna_pivote):
2     columna_cr = self.tableau[1:, -1]
3     columna_pivote_valores = self.tableau[1:, columna_pivote]
4
5     cocientes_validos = []
6
7     for i in range(self.num_restricciones):
8         valor_cr = columna_cr[i]
9         valor_columna = columna_pivote_valores[i]
10
11         if valor_cr < 0 or valor_columna <= 0:
12             continue
13
14         cociente = abs(valor_cr / valor_columna)
15
16         cocientes_validos.append((cociente, i))
17
18     if not cocientes_validos:
19         raise ValueError(
20             "No hay fila pivote válida (problema no acotado, o se requieren "
21             "variables artificiales / método de dos fases para esta tabla)."
22         )
23
24     _, indice_fila = min(cocientes_validos)
25
26     return indice_fila + 1

```

Transformación matricial (_ejecutar_pivote):

Efectúa la eliminación de Gauss-Jordan para hacer unitario el elemento pivote y anular sus coeficientes restantes, modificando el vector de variables básicas.

```

1 def _ejecutar_pivote(self, fila_pivote, columna_pivote):
2     self.variables_basicas[fila_pivote - 1] = columna_pivote
3
4     self.tableau[fila_pivote, :] /= self.tableau[fila_pivote, columna_pivote]
5
6     for i in range(self.tableau.shape[0]):
7         if i != fila_pivote:
8             self.tableau[i, :] -= self.tableau[i, columna_pivote] * self.tableau[fila_pivote, :]
9

```

3.4.3 Método Simplex Gran M (Minimización)

Para abordar modelos con restricciones de tipo mayor o igual ($\geq$) o igualdad ($=$), así como problemas de minimización, el sistema implementa el Método de la Gran M dentro de la clase *SimplexModel*. Para garantizar una base inicial factible, las restricciones de tipo $\geq$ incorporan variables de holgura con signo negativo (-1) junto con variables artificiales (+1), mientras que las restricciones de igualdad ($=$) introducen únicamente variables artificiales (+1). Las variables de holgura estándar (+1) se reservan para restricciones de tipo $\leq$.

A continuación se detalla el funcionamiento de los métodos principales encargados de orquestar este proceso:

Sistema de Solución de Problemas de Programación Lineal

Construcción del Tableau Inicial (_armar_tabla_inicial): Estructura la matriz del tableau asignando los vectores de costos originales y las penalizaciones para las variables artificiales (+M en minimización, -M en maximización). Para mantener la precisión algebraica sin sufrir por desbordamientos numéricos, los coeficientes del tableau se dividen simbólicamente o estructuralmente en dos componentes independientes: la parte numérica constante y la parte asociada al parámetro M.

```

1 def _armar_tabla_inicial(self, restrictions):
2     for i in range(self.num_variables):
3         self.nombres.append(f"x{i + 1}")
4         self.costo_numero[i] = self.objective_coefficients[i]
5
6     for i in range(self.num_holguras_o_excesos):
7         self.nombres.append(f"s{i + 1}")
8
9     for i in range(self.num_artificiales):
10        self.nombres.append(f"A{i + 1}")
11
12    self.nombres.append("RHS")
13
14    signo_de_M = 1 if self.optimization_type != "max" else -1
15    columna_holgura = self.num_variables
16    columna_artificial = self.num_variables + self.num_holguras_o_excesos
17
18    for i in range(self.num_artificiales):
19        self.costo_M[columna_artificial + i] = signo_de_M
20
21    for fila, (coeficientes, operador, termino_independiente) in enumerate(restrictions):
22        coeficientes = list(coeficientes)
23
24        if termino_independiente < 0:
25            coeficientes = [-c for c in coeficientes]
26            termino_independiente = -termino_independiente
27            if operador == "<=":
28                operador = ">="
29            elif operador == ">=":
30                operador = "<="
31
32        for j in range(self.num_variables):
33            self.tableau[fila, j] = coeficientes[j]
34        self.tableau[fila, -1] = termino_independiente
35
36        if operador == "<=":
37            self.tableau[fila, columna_holgura] = 1
38            self.variables_basicas[fila] = columna_holgura
39            columna_holgura += 1
40
41        elif operador == ">=":
42            self.tableau[fila, columna_holgura] = -1
43            self.tableau[fila, columna_artificial] = 1
44            self.variables_basicas[fila] = columna_artificial
45            columna_holgura += 1
46            columna_artificial += 1
47
48        else:
49            raise ValueError(f"Operador de restricción no soportado: {operador}")

```

Figura de código 2: Método Gran M - Construcción del Tableau Inicial y Separación de Componentes Numérica y M (_armar_tabla_inicial)

Sistema de Solución de Problemas de Programación Lineal

Cálculo de Z_j y Costos Reducidos ($_{calcular_zj}$ y $_{calcular_cj_menos_zj}$): El método $_{calcular_zj}$ evalúa la suma del producto vectorial entre los costos de las variables básicas (C_B) y las columnas de la matriz tecnológica, devolviendo tanto la contribución numérica como la de M . Seguidamente, $_{calcular_cj_menos_zj}$ determina los indicadores de entrada ($C_j - Z_j$) restando Z_j del costo original C_j para cada variable no básica.

Selección de Variable Entrante ($_{elegir_columna_pivote}$): Evalúa el criterio de optimalidad examinando las columnas. Primero compara los coeficientes dominantes de M ; en caso de empate o ausencia de M , pasa a evaluar los componentes numéricos. En problemas de minimización, selecciona la columna con el valor $C_j - Z_j$ más negativo (o más positivo en maximización) para determinar la variable que ingresa a la base.

```

1 def _elegir_columna_pivote(self, diferencia_numero, diferencia_M):
2     factor = 1 if self.optimization_type != "max" else -1
3
4     columna_elegida = None
5     mejor_parte_M = None
6     mejor_parte_numero = None
7
8     for columna in range(len(diferencia_numero)):
9         valor_M = factor * diferencia_M[columna]
10        valor_numero = factor * diferencia_numero[columna]
11
12        es_mejor = columna_elegida is None
13        if not es_mejor:
14            if valor_M < mejor_parte_M:
15                es_mejor = True
16            elif valor_M == mejor_parte_M and valor_numero < mejor_parte_numero:
17                es_mejor = True
18
19        if es_mejor:
20            columna_elegida = columna
21            mejor_parte_M = valor_M
22            mejor_parte_numero = valor_numero
23
24        if mejor_parte_M > 0 or (mejor_parte_M == 0 and mejor_parte_numero >= 0):
25            return None
26
27    return columna_elegida
28

```

Selección de Variable Saliente ($_{elegir_fila_pivote}$): Aplica la prueba del cociente mínimo dividiendo los elementos de la columna del lado derecho (RHS) entre los elementos estrictamente positivos (> 0) de la columna pivote elegida. La fila con el menor cociente no negativo señala la variable que abandona la base.

```

1 def _elegir_fila_pivote(self, columna_pivote):
2     fila_elegida = None
3     menor_razon = None
4
5     for fila in range(self.num_restricciones):
6         valor_columna = self.tableau[fila, columna_pivote]
7         if valor_columna > 0:
8             razon = self.tableau[fila, -1] / valor_columna
9             if menor_razon is None or razon < menor_razon:
10                 menor_razon = razon
11                 fila_elegida = fila
12
13     if fila_elegida is None:
14         raise ValueError("El problema no está acotado (solución infinita).")
15
16     return fila_elegida

```

*Actualización del Tableau (*_pivotear*):* Realiza las operaciones elementales de renglón mediante la eliminación de Gauss-Jordan. Normaliza la fila pivote dividiéndola por el elemento pivote e introduce ceros en las demás filas de la columna pivote, actualizando además el vector de variables básicas.

```

1 def _pivotear(self, fila_pivote, columna_pivote):
2     self.variables_basicas[fila_pivote] = columna_pivote
3
4     valor_pivote = self.tableau[fila_pivote, columna_pivote]
5     for columna in range(self.tableau.shape[1]):
6         self.tableau[fila_pivote, columna] /= valor_pivote
7
8     for fila in range(self.tableau.shape[0]):
9         if fila != fila_pivote:
10             factor = self.tableau[fila, columna_pivote]
11             for columna in range(self.tableau.shape[1]):
12                 self.tableau[fila, columna] -= factor * self.tableau[fila_pivote, columna]

```

*Bucle de Optimización y Validaciones (*resolver*):* Ejecuta el bucle principal de iteraciones mientras existan candidatos a ingresar según el criterio de $C_j - Z_j$. Al alcanzar la condición de parada (optimalidad), realiza la verificación final de factibilidad: comprueba que ninguna variable artificial permanezca en la base con un valor strictly mayor a cero (> 0). Si alguna permanece positiva, se concluye que el problema carece de región factible.

```

1 def resolver(self):
2     while True:
3         zj_numero, zj_M = self._calcular_zj(self.tableau, self.variables_basicas)
4         diferencia_numero, diferencia_M = self._calcular_cj_menos_zj(zj_numero, zj_M)
5
6         columna_pivote = self._elegir_columna_pivote(diferencia_numero, diferencia_M)
7         if columna_pivote is None:
8             break
9
10        fila_pivote = self._elegir_fila_pivote(columna_pivote)
11        self._guardar_estado(pivote=(fila_pivote, columna_pivote))
12        self._pivotear(fila_pivote, columna_pivote)
13
14        self._guardar_estado(pivote=None)
15
16        primera_columna_artificial = self.num_variables + self.num_holguras_o_excesos
17        for fila in range(self.num_restricciones):
18            variable_basica = self.variables_basicas[fila]
19            if variable_basica >= primera_columna_artificial and self.tableau[fila, -1] > 0:
20                raise ValueError("El problema no tiene región factible.")
21
22        return self.historial

```

El criterio de parada valida que no queden coeficientes que puedan mejorar la función objetivo y asegura que todas las variables artificiales se hayan reducido a cero en la solución óptima final.

IV. RESULTADOS

Para validar el funcionamiento del sistema, se resolvió el siguiente problema:

“Se dispone de 600 g de un determinado fármaco para elaborar pastillas grandes y pequeñas. Las grandes pesan 40 g y las pequeñas 30 g. Se necesitan al menos tres pastillas grandes, y al menos el doble de pequeñas que de las grandes. Cada pastilla grande proporciona un beneficio de \$2 y la pequeña de \$1. ¿Cuántas pastillas se han de elaborar de cada clase para que el beneficio sea máximo?”

El modelo matemático es:

Variables:

x_1 : número de pastillas grandes.

x_2 : número de pastillas pequeñas.

Función objetivo (maximizar):

$$Z = 2x_1 + x_2$$

Restricciones:

$$40x_1 + 30x_2 \leq 600 \text{ (peso total)} \quad x_1 \geq 3 \text{ (mínimo grandes)} \quad x_2 \geq 2x_1 \text{ (al menos el doble de pequeñas)} \quad x_1, x_2 \geq 0$$

4.1 Solución mediante Método Gráfico

Al ejecutar el software con el método gráfico, se obtiene la región factible y los vértices calculados. Los vértices factibles son:

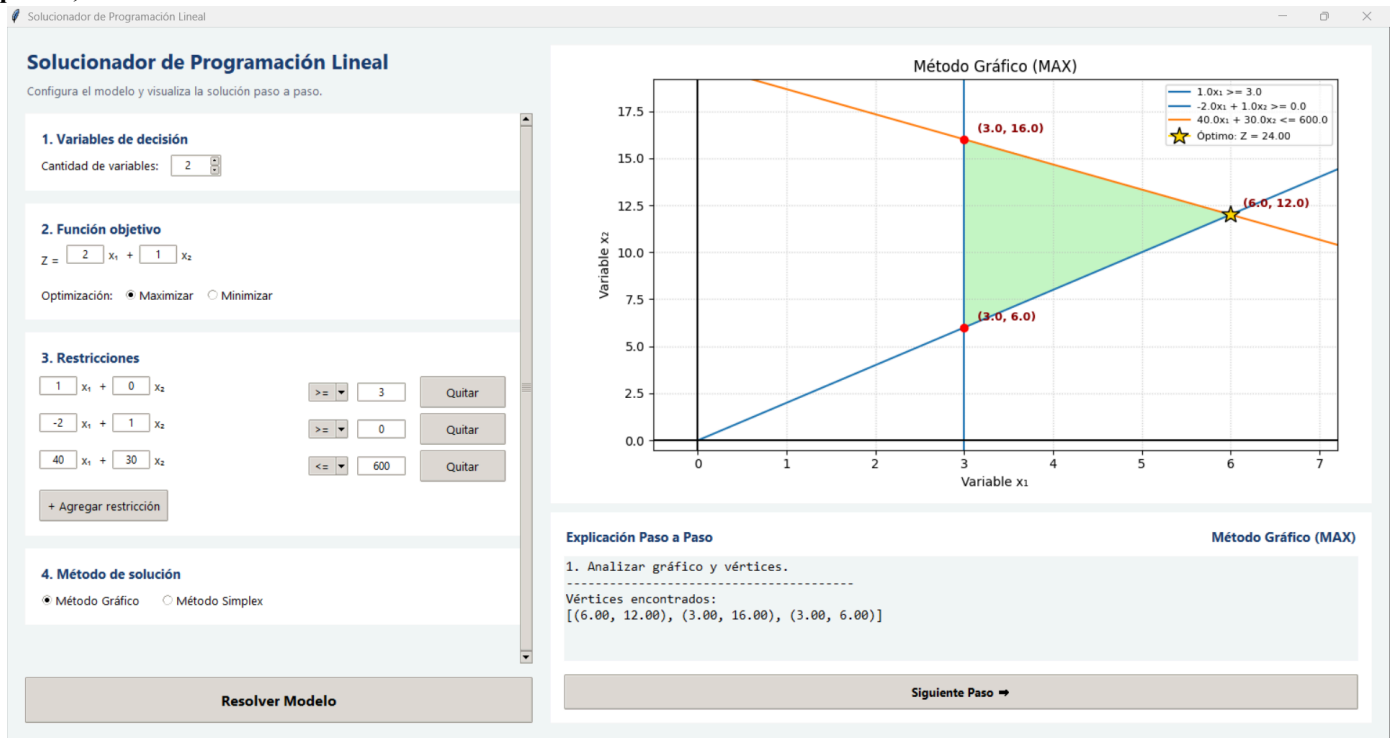
- (6)
- (16)
- (12)

Evaluando la función objetivo:

- $Z(6) = 2(3) + 6 = 12$
- $Z(16) = 2(3) + 16 = 22$
- $Z(12) = 2(6) + 12 = 24$

Sistema de Solución de Problemas de Programación Lineal

El valor máximo es 24, alcanzado en $(x_2) = (12)$. Por lo tanto, la solución óptima es fabricar **6 pastillas grandes y 12 pequeñas**, con un beneficio de **\$24**.



Captura 1: Vista de la interfaz gráfica mostrando la región factible sombreada, las rectas de las restricciones, los vértices marcados y el punto óptimo señalado con una estrella.

4.2 Solución mediante Método Simplex Común

Al seleccionar el método Simplex común, el software muestra de forma interactiva las iteraciones del algoritmo tabular. Para el problema planteado, el sistema aplica las transformaciones correspondientes hasta converger a la solución óptima deseada.

El tableau final muestra que todas las variables no básicas tienen coeficientes no negativos en la fila Z (para maximización), confirmando la optimalidad.

Sistema de Solución de Problemas de Programación Lineal

Solucionador de Programación Lineal

Configura el modelo y visualiza la solución paso a paso.

1. Variables de decisión
Cantidad de variables: 2

2. Función objetivo
 $Z = 2x_1 + 1x_2$
Optimización: ☒ Maximizar ☐ Minimizar

3. Restricciones
 $1x_1 + 0x_2 \geq 3$ Quitar
 $-2x_1 + 1x_2 \geq 0$ Quitar
 $40x_1 + 30x_2 \leq 600$ Quitar
 + Agregar restricción

4. Método de solución
☐ Método Gráfico ☒ Simplex Común
☐ Simplex Dos Fases ☐ Simplex Gran M

Resolver Modelo

Método Simplex — Solución óptima

	x1	x2	s1	s2	s3	RHS
Z	0.00	0.00	0.00	0.20	0.04	24.00
s1	0.00	0.00	1.00	0.30	0.01	3.00
x1	1.00	0.00	0.00	0.30	0.01	6.00
x2	0.00	1.00	0.00	-0.40	0.02	12.00

Explicación Paso a Paso Método Simplex

Iteración 2 — Método Simplex Común

Solución óptima alcanzada.

Siguiente Paso ➡

Captura 2: Iteraciones del Simplex, mostrando el tableau con el pivote resaltado y la variable que entra/sale, junto con el texto explicativo.

4.3 Solución mediante Método Simplex Gran M

Para validar el comportamiento frente a restricciones del tipo $\geq$, el problema se resolvió mediante el Método Simplex Gran M. Durante la ejecución, el algoritmo incorporó las variables artificiales y aplicó las penalizaciones correspondientes en la función objetivo, permitiendo iterar y eliminar dichas variables hasta alcanzar nuevamente la solución óptima esperada.

Solucionador de Programación Lineal

Configura el modelo y visualiza la solución paso a paso.

Cantidad de variables: 2

2. Función objetivo
 $Z = 2x_1 + 1x_2$
Optimización: ☒ Maximizar ☐ Minimizar

3. Restricciones
 $1x_1 + 0x_2 \geq 3$ Quitar
 $-2x_1 + 1x_2 \geq 0$ Quitar
 $40x_1 + 30x_2 \leq 600$ Quitar
 + Agregar restricción

4. Método de solución
☐ Método Gráfico ☐ Simplex Común
☐ Simplex Dos Fases ☒ Simplex Gran M

Resolver Modelo

Método Gran M — Solución óptima

Cb	VB	x1	x2	s1	s2	s3	A1	A2	RHS
	Cj	2	1	0	0	0	-M	-M	
2	x1	1.00	0.00	0.00	0.30	0.01	0.00	-0.30	6.00
1	x2	0.00	1.00	0.00	-0.40	0.02	0.00	0.40	12.00
0	s1	0.00	0.00	1.00	0.30	0.01	-1.00	-0.30	3.00
	Zj	2	1	0	0.2	0.04	0	-0.2	24
	Cj - Zj	0	0	0	-0.2	-0.04	-M	0.2 - M	

Explicación Paso a Paso Método Gran M

Iteración 3 — Método de la Gran M

Solución óptima alcanzada.

Siguiente Paso ➡

Captura 3: Iteración del Método Simplex Gran M, mostrando la tabla final (tableau) con la solución óptima alcanzada tras la eliminación de las variables artificiales de la base.

V. COCLUSIONES

El software desarrollado constituye una herramienta didáctica eficaz para la enseñanza de la Programación Lineal. La adopción de la arquitectura MVC y la separación entre los modelos de solución y la vista permiten una fácil extensión y mantenimiento. Los métodos gráfico, Simplex Común y Gran M han sido implementados siguiendo rigurosamente los algoritmos clásicos, y su correcto funcionamiento ha sido verificado mediante la resolución de problemas del taller.

Fortalezas identificadas:

- Visualización paso a paso que facilita la comprensión de los algoritmos.
- Manejo de problemas con restricciones mixtas ($\leq$, $\geq$, $=$) y optimización tanto maximización como minimización.
- Modularidad que permite agregar nuevos métodos de solución en el futuro.

Áreas de mejora:

- El método gráfico está limitado a dos variables; se podría ampliar a tres con visualización 3D.
- La función de graficación del tableau (graficar_tableau) podría integrarse directamente en la clase para mayor cohesión.
- Se recomienda agregar validaciones más exhaustivas para detectar problemas no acotados o infactibles.

En general, el sistema cumple con los objetivos planteados y proporciona una base sólida para el aprendizaje interactivo de la Programación Lineal.

VI. REFERENCIAS

1. Taha, H. A. (2017). *Investigación de Operaciones*. Pearson Educación.
2. Winston, W. L. (2004). *Investigación de Operaciones: Aplicaciones y Algoritmos*. Thomson.
3. Documentación de Python: <https://docs.python.org/3/>
4. NumPy: <https://numpy.org/doc/stable/>
5. Matplotlib: <https://matplotlib.org/stable/contents.html>